



PIMPRI CHINCHWAD EDUCATION TRUST'S.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)

B. TECH

Year: 2026 – 27

Semester: VII

Name of the Student: Swayam Mandhani

PRN: 123B1B184

Department: Computer Engineering

Division: C, **Batch** – C2

Course: DevOps

Course Code: BCE27PE01

Assignment No – 8

Title:

Explore Kubernetes Objects and Services and Demonstrate Configuration Management Using Ansible.

Aim:

To study important Kubernetes objects and networking services and demonstrate automated configuration management using Ansible.

Objectives:

- Understand and create important Kubernetes objects including Pod, Deployment, Service, Namespace, ConfigMap, Secret, Persistent Volume and Persistent Volume Claim.
- Study and demonstrate Kubernetes Service types: ClusterIP, NodePort, LoadBalancer and ExternalName.
- Configure WSL Ubuntu as an Ansible control node.
- Create Linux managed nodes and establish SSH-based Ansible connectivity.
- Use an Ansible inventory and playbook to install and configure Nginx on multiple managed nodes.
- Deploy a custom web page using Ansible and verify the configuration.
- Demonstrate repeatable/idempotent configuration management by executing the playbook again.

Theory:

Kubernetes Objects

Object	Purpose
Pod	The smallest deployable unit in Kubernetes. A Pod contains one or more containers that share networking and storage resources.
Deployment	Manages replicated application Pods and provides declarative updates and rollout management.
Service	Provides stable network access to a set of Pods using selectors, independent of individual Pod IP changes.
Namespace	Provides logical isolation and organization of Kubernetes resources within a cluster.
ConfigMap	Stores non-sensitive configuration data separately from application/container images.
Secret	Stores sensitive configuration data such as usernames or passwords. Dummy credentials were used for this practical.
Persistent Volume (PV)	Represents storage provisioned for Kubernetes workloads and exists independently of an individual Pod.
Persistent Volume Claim (PVC)	A request for storage made by a workload. A PVC can bind to a compatible Persistent Volume.

Kubernetes Service Types

- ClusterIP – Provides internal cluster access to a Service.
- NodePort – Exposes the Service on a port on each Kubernetes node.
- LoadBalancer – Requests external load-balancer exposure where the cluster/cloud environment provides a load-balancer implementation.
- ExternalName – Maps a Kubernetes Service name to an external DNS name.

Ansible

Ansible is an agentless automation and configuration-management tool. In this practical, WSL Ubuntu was used as the control node. Two Ubuntu Docker containers were used as managed Linux nodes. Ansible connected to the nodes through SSH and used a playbook to install Nginx and curl, start/configure Nginx, and deploy a custom HTML page.

- Control Node – The machine from which Ansible commands and playbooks are executed.
- Managed Nodes – Remote systems controlled by Ansible.
- Inventory – Defines and groups managed hosts and their connection information.
- Playbook – YAML automation definition containing plays and tasks.
- Module – Reusable unit of Ansible functionality used to perform operations.
- Idempotency – Repeated execution should converge on the desired state without unnecessary changes.

Tools and Environment:

Component	Configuration
Operating System / Control Node	Windows with WSL 2 Ubuntu
Kubernetes Environment	Docker Desktop Kubernetes
Kubernetes CLI	kubectl
Container Runtime	Docker Desktop
Ansible Control Node	WSL Ubuntu
Managed Nodes	Two Ubuntu 24.04 Docker containers
Web Server	Nginx
Automation	Ansible over SSH
Editor	Visual Studio Code

System / Practical Architecture:

The practical combines two local environments. Docker Desktop Kubernetes is used for Kubernetes objects and Services, while WSL Ubuntu acts as the Ansible control node for two Linux managed nodes.

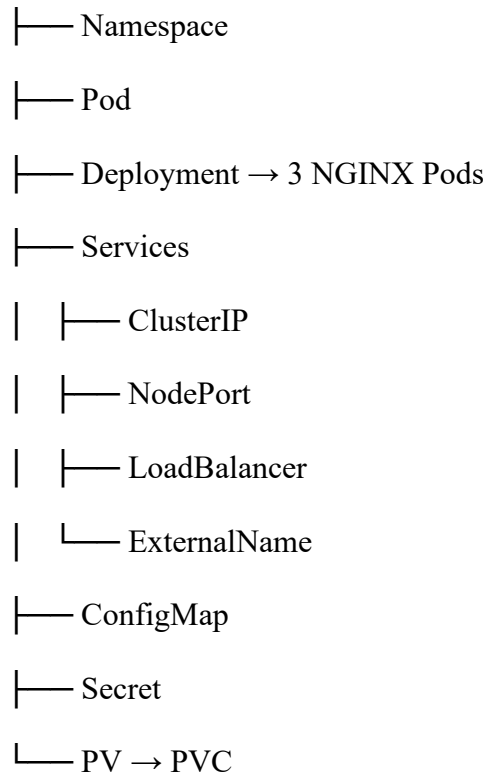
Kubernetes Part

Windows / Docker Desktop

|

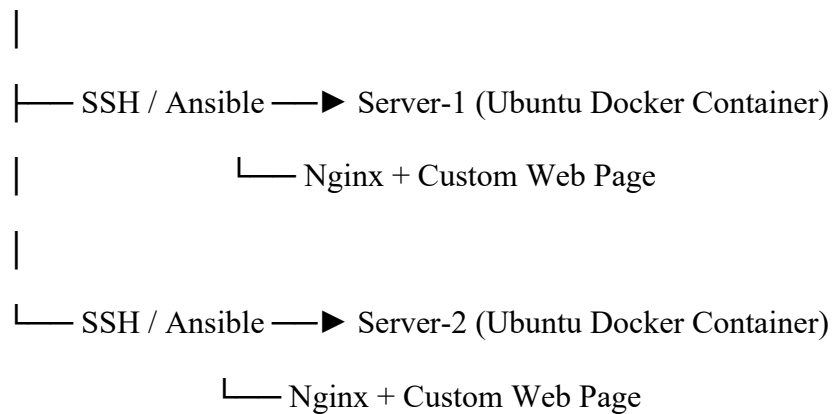


Kubernetes Cluster



Ansible Part

WSL Ubuntu (Control Node)



Part A – Kubernetes Objects and Services:

Namespace

apiVersion: v1

kind: Namespace

metadata:

name: assignment8

The namespace `assignment8` was created to logically group the resources used in this practical.

Pod

apiVersion: v1

kind: Pod

metadata:

name: nginx-pod

namespace: assignment8

labels:

app: nginx-pod

spec:

containers:

- name: nginx

image: nginx:latest

ports:

- containerPort: 80

kubectl apply -f pod.yaml

kubectl get pods -n assignment8

Deployment

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

namespace: assignment8

spec:

replicas: 3

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx:latest

ports:

- containerPort: 80

kubectl apply -f deployment.yaml

kubectl get deployments -n assignment8

kubectl get pods -n assignment8

The Deployment was verified with three available NGINX replicas.

NodePort Service

apiVersion: v1

kind: Service

metadata:

name: nginx-nodeport

namespace: assignment8

spec:

type: NodePort

selector:

app: nginx

ports:

- port: 80

targetPort: 80

kubectl apply -f nodeport-service.yaml

kubectl get service -n assignment8

ConfigMap

apiVersion: v1

kind: ConfigMap

metadata:

name: app-config

namespace: assignment8

data:

APP_ENV: production

APP_NAME: Assignment8-Nginx

Secret

apiVersion: v1

kind: Secret

metadata:

name: db-secret

namespace: assignment8

type: Opaque

stringData:

username: admin

password: admin123

Dummy credentials were used only for demonstration. Secret values were not exposed in the verification screenshots.

Persistent Volume and Persistent Volume Claim

apiVersion: v1

kind: PersistentVolume

metadata:

name: assignment8-pv

spec:

capacity:

storage: 1Gi

```
accessModes:
  - ReadWriteOnce
persistentVolumeReclaimPolicy: Retain
storageClassName: ""
hostPath:
  path: /tmp/assignment8-data
```

```
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: assignment8-pvc
  namespace: assignment8
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ""
  resources:
    requests:
      storage: 1Gi
```

The static PV/PVC configuration was adjusted so that the claim could bind directly to the assignment PV. The final verification showed the PV and PVC in Bound state.

Kubernetes Service Types

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-clusterip
  namespace: assignment8
spec:
  type: ClusterIP
  selector:
```

```
  app: nginx
ports:
  - port: 80
    targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-loadbalancer
  namespace: assignment8
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: external-service
  namespace: assignment8
spec:
  type: ExternalName
  externalName: example.com
kubectl apply -f service-types.yaml
kubectl get services -n assignment8
```

The final local cluster output showed all four required Service types. The Docker Desktop environment assigned an external IP to the LoadBalancer Service.

Key Kubernetes Verification Commands

```
kubectrl get all -n assignment8
```

```
kubectrl get configmap,secrets,pvc -n assignment8
```

```
kubectrl get pv
```

Part B – Ansible Configuration Management Case Study:

Control Node

WSL Ubuntu was used as the Ansible control node. Ansible was installed inside WSL and verified using the Ansible version command.

```
wsl --status
```

```
wsl -l -v
```

```
uname -a
```

```
python3 --version
```

```
ansible --version
```

Managed-Node Image

FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \

apt-get install -y openssh-server sudo python3 nginx && \

rm -rf /var/lib/apt/lists/*

RUN useradd -m -s /bin/bash ansible && \

echo "ansible:ansible123" | chpasswd && \

echo "ansible ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/ansible && \

chmod 440 /etc/sudoers.d/ansible

RUN mkdir -p /run/sshd && \

sed -i 's/^#?PasswordAuthentication.*/PasswordAuthentication yes/' /etc/ssh/sshd_config && \

sed -i 's/^#?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D", "-e"]

Two containers were created from the image and exposed through local SSH ports 2221 and 2222.

Inventory

[webservers]

server-1 ansible_host=127.0.0.1 ansible_port=2221 ansible_user=ansible

server-2 ansible_host=127.0.0.1 ansible_port=2222 ansible_user=ansible

Connectivity Test

ansible all -i inventory.ini -m ping -k

Both managed nodes returned SUCCESS with ping: pong, confirming SSH connectivity and Ansible reachability.

Nginx Configuration Playbook

- name: Configure Web Servers

hosts: webservers

become: yes

tasks:

- name: Install Nginx and curl

ansible.builtin.apt:

name:

- nginx

- curl

state: present

update_cache: yes

- name: Start and enable Nginx

ansible.builtin.service:

name: nginx

state: started

enabled: yes

- name: Deploy custom web page

ansible.builtin.copy:

content: |

<html>

<head>

<title>Ansible Assignment 8</title>

</head>

<body>

<h1>Web Server Configured Using Ansible</h1>

<p>Server configured automatically using Ansible.</p>

</body>

</html>

dest: /var/www/html/index.html

mode: '0644'

Playbook Execution

```
ansible-playbook -i inventory.ini install-nginx.yml --syntax-check
```

```
ansible-playbook -i inventory.ini install-nginx.yml -k
```

The playbook installed/configured Nginx and curl and deployed the custom web page on both managed nodes. The final execution completed without failed tasks.

Nginx Verification

```
ansible server-1 -i inventory.ini -m shell -a "curl -s http://localhost" -k
```

```
ansible server-2 -i inventory.ini -m shell -a "curl -s http://localhost" -k
```

```
ansible webservers -i inventory.ini -m shell -a "nginx -v" -k
```

```
ansible webservers -i inventory.ini -m shell -a "pgrep -a nginx" -k
```

The custom page text “Web Server Configured Using Ansible” was returned from both servers. Nginx version/process verification was also completed.

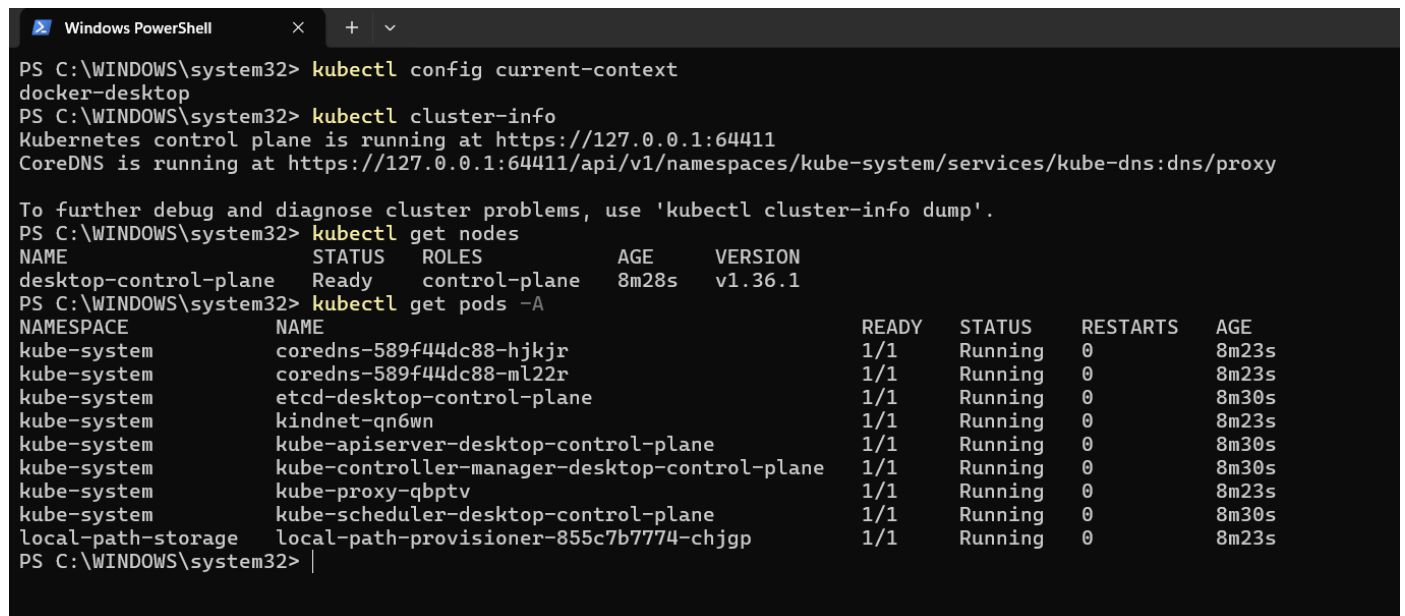
Idempotency Verification

```
ansible-playbook -i inventory.ini install-nginx.yml -k
```

The playbook was executed again after the desired configuration had already been applied. The repeat execution was used to demonstrate Ansible's idempotent configuration-management behavior

Output

The following figures provide evidence of Kubernetes cluster verification, creation and verification of Kubernetes objects and Services, Ansible control-node configuration, managed-node connectivity, playbook execution, Nginx configuration, web-page deployment, process verification, and idempotency.



```
Windows PowerShell
PS C:\WINDOWS\system32> kubectl config current-context
docker-desktop
PS C:\WINDOWS\system32> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:64411
CoreDNS is running at https://127.0.0.1:64411/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\WINDOWS\system32> kubectl get nodes
NAME                STATUS    ROLES    AGE     VERSION
desktop-control-plane Ready    control-plane  8m28s   v1.36.1
PS C:\WINDOWS\system32> kubectl get pods -A
NAMESPACE   NAME                                     READY   STATUS    RESTARTS   AGE
kube-system   coredns-589f44dc88-hjkjr              1/1     Running   0           8m23s
kube-system   coredns-589f44dc88-ml22r              1/1     Running   0           8m23s
kube-system   etcd-desktop-control-plane            1/1     Running   0           8m30s
kube-system   kindnet-qn6wn                         1/1     Running   0           8m23s
kube-system   kube-apiserver-desktop-control-plane   1/1     Running   0           8m30s
kube-system   kube-controller-manager-desktop-control-plane 1/1     Running   0           8m30s
kube-system   kube-proxy-qbptv                      1/1     Running   0           8m23s
kube-system   kube-scheduler-desktop-control-plane   1/1     Running   0           8m30s
local-path-storage local-path-provisioner-855c7b7774-chjgp 1/1     Running   0           8m23s
PS C:\WINDOWS\system32> |
```

Figure 1: Kubernetes Cluster Verification

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl apply -f namespace.yaml
namespace/assignment8 created
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get namespaces
NAME                STATUS    AGE
assignment8         Active   9s
default              Active  11m
kube-node-lease      Active  11m
kube-public          Active  11m
kube-system          Active  11m
local-path-storage   Active  11m
```

Figure 2: Namespace Created

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl apply -f pod.yaml
pod/nginx-pod unchanged
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get all -n assignment8
NAME                READY   STATUS    RESTARTS   AGE
pod/nginx-pod       1/1     Running   0           27s
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get pods -n assignment8 -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE                NOMINATED NODE   READINESS GATES
nginx-pod           1/1     Running   0           33s   10.244.0.5      desktop-control-plane   <none>            <none>
```

Figure 3: Pod Created

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl apply -f deployment.yaml
deployment.apps/nginx-deployment created
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get deployments -n assignment8
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    3/3     3             3           20s
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get pods -n assignment8
NAME                READY   STATUS    RESTARTS   AGE
nginx-deployment-59c4c87bc6-gbk79  1/1     Running   0           29s
nginx-deployment-59c4c87bc6-kgf7n  1/1     Running   0           29s
nginx-deployment-59c4c87bc6-vmtc9  1/1     Running   0           29s
nginx-pod           1/1     Running   0           6m36s
```

Figure 4: Deployment Created

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl apply -f nodeport-service.yaml
service/nginx-nodeport created
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get service -n assignment8
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
nginx-nodeport      NodePort    10.96.180.191    <none>            80:31678/TCP     9s
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get service nginx-nodeport -n assignment8
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
nginx-nodeport      NodePort    10.96.180.191    <none>            80:31678/TCP     16s
```

Figure 5: NodePort Service

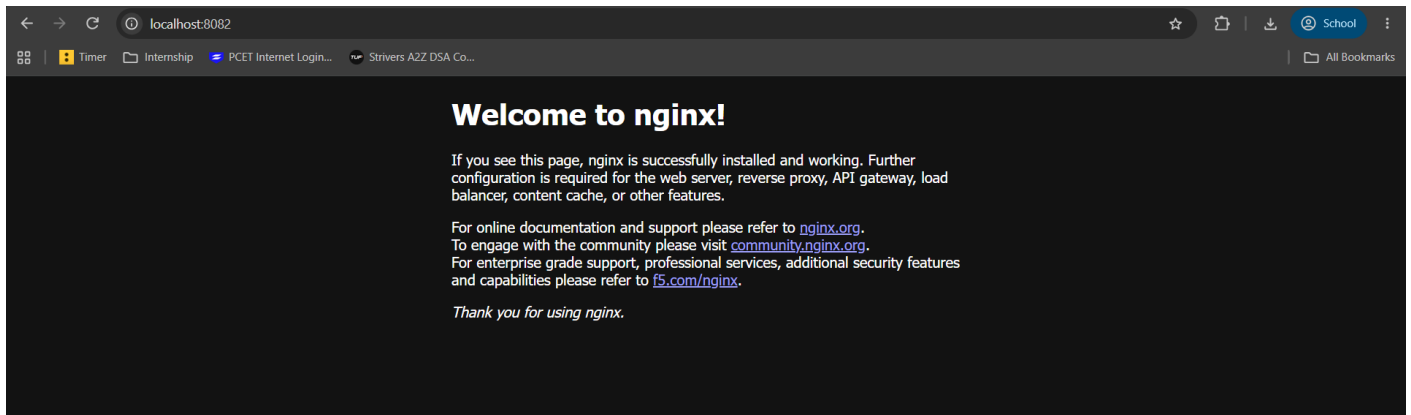


Figure 6: NGINX Service Verification

```

Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl apply -f configmap.yaml
configmap/app-config created
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get configmap -n assignment8
NAME          DATA      AGE
app-config    2          8s
kube-root-ca.crt 1          12m
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl describe configmap app-config -n assignment8
Name:         app-config
Namespace:    assignment8
Labels:       <none>
Annotations:  <none>

Data
====
APP_ENV:
----
production

APP_NAME:
----
Assignment8-Nginx

BinaryData
====

Events: <none>
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> |

```

Figure 7: ConfigMap Created

```

Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl apply -f secret.yaml
secret/db-secret created
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get secrets -n assignment8
NAME          TYPE      DATA      AGE
db-secret     Opaque    2          9s
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get secrets -n assignment8
NAME          TYPE      DATA      AGE
db-secret     Opaque    2          14s
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> |

```

Figure 8: Secret Created

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl apply -f pv-pvc.yaml
persistentvolume/assignment8-pv configured
persistentvolumeclaim/assignment8-pvc created
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM                STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE
assignment8-pv 1Gi       RWO           Retain          Bound   assignment8/assignment8-pvc  <unset>
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get pvc -n assignment8
NAME          STATUS  VOLUME          CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
assignment8-pvc Bound   assignment8-pv  1Gi       RWO           <unset>        <unset>                 22s
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> |
```

Figure 9: Persistent Volume and Claim

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl apply -f service-types.yaml
service/nginx-clusterip created
service/nginx-loadbalancer created
service/external-service created
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> kubectl get services -n assignment8
NAME          TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
external-service  ExternalName  <none>           example.com       <none>           10s
nginx-clusterip  ClusterIP     10.96.12.26      <none>            80/TCP           10s
nginx-loadbalancer LoadBalancer  10.96.111.217    172.18.0.5       80:30981/TCP     10s
nginx-nodeport   NodePort      10.96.180.191    <none>            80:31678/TCP     9m32s
PS D:\Projects\DevOps Github\Assignment-8\kubernetes> |
```

Figure 10: Kubernetes Service Types

```
PS D:\Projects\DevOps Github\Assignment-8\ansible> wsl --status
Default Distribution: Ubuntu
Default Version: 2
WSL1 is not supported with your current machine configuration.
Please enable the "Windows Subsystem for Linux" optional component to use WSL1.
PS D:\Projects\DevOps Github\Assignment-8\ansible> wsl -l -v
NAME      STATE      VERSION
* Ubuntu  Stopped    2
docker-desktop Running    2
PS D:\Projects\DevOps Github\Assignment-8\ansible> wsl -d Ubuntu
swayam@SWAYAM:/mnt/d/Projects/DevOps Github/Assignment-8/ansible$ uname -a
Linux SWAYAM 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
swayam@SWAYAM:/mnt/d/Projects/DevOps Github/Assignment-8/ansible$ python3 --version
Python 3.14.4
swayam@SWAYAM:/mnt/d/Projects/DevOps Github/Assignment-8/ansible$ which python3
/usr/bin/python3
swayam@SWAYAM:/mnt/d/Projects/DevOps Github/Assignment-8/ansible$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [681 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [543 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [162 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [98.0 kB]
Get:9 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [311 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [99.5 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [190 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [457 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [90.8 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [11.9 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse Translation-en [3276 B]
Get:16 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1060 B]
Get:17 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [127 kB]
Get:18 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:19 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [187 kB]
Get:20 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [59.3 kB]
Get:21 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.4 kB]
Get:22 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [443 kB]
Get:23 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [87.8 kB]
Get:24 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64 Packages [10.8 kB]
Get:25 http://security.ubuntu.com/ubuntu resolute-security/multiverse Translation-en [2844 B]
Fetched 4069 kB in 4s (924 kB/s)
118 packages can be upgraded. Run 'apt list --upgradable' to see them.
swayam@SWAYAM:/mnt/d/Projects/DevOps Github/Assignment-8/ansible$ |
```

Figure 11: Ansible Control Node

```

swayam@SWAYAM:/mnt/d/Projects/DevOps Github/Assignment-8/ansible$ ansible --version
ansible [core 2.20.1]
  config file = None
  configured module search path = ['/home/swayam/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/swayam/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.14.4 (main, Jun 18 2026, 14:25:02) [GCC 15.2.0] (/usr/bin/python3)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
swayam@SWAYAM:/mnt/d/Projects/DevOps Github/Assignment-8/ansible$ ansible-inventory --version
ansible-inventory [core 2.20.1]
  config file = None
  configured module search path = ['/home/swayam/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/swayam/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible-inventory
  python version = 3.14.4 (main, Jun 18 2026, 14:25:02) [GCC 15.2.0] (/usr/bin/python3)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
swayam@SWAYAM:/mnt/d/Projects/DevOps Github/Assignment-8/ansible$ |

```

Figure 12: Ansible Version

```

swayam@SWAYAM: ~/assignment8-ansible$ docker run -d \
--name assignment8-server1 \
-p 127.0.0.1:2221:22 \
assignment8-ansible-node
b9767347ad279b49b45f027ca21882808f4cae9c7eda2a2f5350da0e6768d2ec
swayam@SWAYAM:~/assignment8-ansible$ docker run -d \
--name assignment8-server2 \
-p 127.0.0.1:2222:22 \
assignment8-ansible-node
f4031b26697364451d85631c3503b2c34d2bbfec51bc8d60d5379a31451896f9
swayam@SWAYAM:~/assignment8-ansible$ docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
f4031b266973	assignment8-ansible-node	"/usr/sbin/sshd -D -e"	5 seconds ago	Up 4 seconds	127.0.0.1:2222->22/tcp	assignment8-server2
b9767347ad27	assignment8-ansible-node	"/usr/sbin/sshd -D -e"	11 seconds ago	Up 11 seconds	127.0.0.1:2221->22/tcp	assignment8-server1
a56343f6d525	mongo:7	"docker-entrypoint.s..."	4 weeks ago	Up 3 minutes (healthy)	27017/tcp	campuscare-mongodb

```

swayam@SWAYAM:~/assignment8-ansible$ |

```

Figure 13: Ansible Managed Servers

```

swayam@SWAYAM:~/assignment8-ansible$ nano inventory.ini
swayam@SWAYAM:~/assignment8-ansible$ cat inventory.ini
[webserver]

server-1 ansible_host=127.0.0.1 ansible_port=2221 ansible_user=ansible
server-2 ansible_host=127.0.0.1 ansible_port=2222 ansible_user=ansible
swayam@SWAYAM:~/assignment8-ansible$ ansible-inventory -i inventory.ini --graph
@all:
|--@ungrouped:
|--@webserver:
| |--server-1
| |--server-2
swayam@SWAYAM:~/assignment8-ansible$ |

```

Figure 14: Ansible Inventory

```
swayam@SWAYAM: ~/assignment8-ansible$ ansible all -i inventory.ini -m ping -k
SSH password:
[WARNING]: Host 'server-1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could
cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more info
rmation.
server-1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host 'server-2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could
cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more info
rmation.
server-2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
swayam@SWAYAM:~/assignment8-ansible$
```

Figure 15: Ansible Ping Success

```
swayam@SWAYAM: ~/assignment8-ansible$ ansible-playbook -i inventory.ini install-nginx.yml --syntax-check
playbook: install-nginx.yml
swayam@SWAYAM:~/assignment8-ansible$
```

Figure 16: Ansible Playbook Syntax Check

```
swayam@SWAYAM:~/assignment8-ansible$ ansible-playbook -i inventory.ini install-nginx.yml -k
SSH password:

PLAY [Configure Web Servers] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'server-1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could
cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more info
rmation.
ok: [server-1]
[WARNING]: Host 'server-2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could
cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more info
rmation.
ok: [server-2]

TASK [Install Nginx] *****
ok: [server-2]
ok: [server-1]

TASK [Start and enable Nginx] *****
changed: [server-2]
changed: [server-1]

TASK [Deploy custom web page] *****
changed: [server-2]
changed: [server-1]

PLAY RECAP *****
server-1      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server-2      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
swayam@SWAYAM:~/assignment8-ansible$
```

Figure 17: Ansible Playbook Success

```
swayam@SWAYAM: ~/assignment8-ansible$ ansible server-1 -i inventory.ini -m shell -a "curl -s http://localhost" -k
SSH password:
[WARNING]: Host 'server-1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could
cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more info
rmation.
server-1 | CHANGED | rc=0 >>
<html>
<head>
  <title>Ansible Assignment 8</title>
</head>
<body>
  <h1>Web Server Configured Using Ansible</h1>
  <p>Server configured automatically using Ansible.</p>
</body>
</html>
swayam@SWAYAM:~/assignment8-ansible$ ansible server-2 -i inventory.ini -m shell -a "curl -s http://localhost" -k
SSH password:
[WARNING]: Host 'server-2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could
cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more info
rmation.
server-2 | CHANGED | rc=0 >>
<html>
<head>
  <title>Ansible Assignment 8</title>
</head>
<body>
  <h1>Web Server Configured Using Ansible</h1>
  <p>Server configured automatically using Ansible.</p>
</body>
</html>
swayam@SWAYAM:~/assignment8-ansible$ |
```

Figure 18: Nginx Ansible Verification

```
swayam@SWAYAM: ~/assignment8-ansible$ ansible webserver -i inventory.ini -m shell -a "pgrep -a nginx" -k
SSH password:
[WARNING]: Host 'server-1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could
cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more info
rmation.
server-1 | CHANGED | rc=0 >>
877 nginx: master process /usr/sbin/nginx
878 nginx: worker process
879 nginx: worker process
880 nginx: worker process
881 nginx: worker process
882 nginx: worker process
883 nginx: worker process
884 nginx: worker process
885 nginx: worker process
886 nginx: worker process
887 nginx: worker process
888 nginx: worker process
889 nginx: worker process
890 nginx: worker process
891 nginx: worker process
892 nginx: worker process
893 nginx: worker process
[WARNING]: Host 'server-2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could
cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more info
rmation.
server-2 | CHANGED | rc=0 >>
875 nginx: master process /usr/sbin/nginx
876 nginx: worker process
877 nginx: worker process
878 nginx: worker process
879 nginx: worker process
880 nginx: worker process
881 nginx: worker process
882 nginx: worker process
883 nginx: worker process
884 nginx: worker process
885 nginx: worker process
886 nginx: worker process
887 nginx: worker process
888 nginx: worker process
889 nginx: worker process
890 nginx: worker process
891 nginx: worker process
892 nginx: worker process
893 nginx: worker process
swayam@SWAYAM:~/assignment8-ansible$ |
```

Figure 19: Nginx Process Verification

```
swayam@SWAYAM: ~/assignment8-ansible$ ansible-playbook -i inventory.ini install-nginx.yml -k
SSH password:

PLAY [Configure Web Servers] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'server-2' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [server-2]
[WARNING]: Host 'server-1' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [server-1]

TASK [Install Nginx and curl] *****
ok: [server-2]
ok: [server-1]

TASK [Start and enable Nginx] *****
ok: [server-2]
ok: [server-1]

TASK [Deploy custom web page] *****
ok: [server-1]
ok: [server-2]

PLAY RECAP *****
server-1      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server-2      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

swayam@SWAYAM:~/assignment8-ansible$ |
```

Figure 20: Ansible Idempotency

Result:

The practical successfully demonstrated the creation and inspection of the required Kubernetes objects and Service types using Docker Desktop Kubernetes. A Pod, Deployment with three replicas, NodePort Service, Namespace, ConfigMap, Secret, Persistent Volume and Persistent Volume Claim were created and verified. ClusterIP, NodePort, LoadBalancer and ExternalName Service types were also demonstrated.

For the Ansible case study, WSL Ubuntu was configured as the control node and two Ubuntu Docker containers were used as managed nodes. SSH connectivity was verified through Ansible, Nginx and curl were installed, Nginx was configured, a custom web page was deployed, and the result was verified from both managed nodes. A repeated playbook execution was used to demonstrate idempotent configuration management.

Learning Outcome:

- Identified the purpose and usage of major Kubernetes objects.
- Differentiated ClusterIP, NodePort, LoadBalancer and ExternalName Service types.
- Created and inspected Kubernetes resources using kubectl and YAML manifests.
- Understood the relationship between Persistent Volume and Persistent Volume Claim.
- Configured WSL Ubuntu as an Ansible control node.

- Created an Ansible inventory and established SSH-based connectivity with multiple managed nodes.
- Used an Ansible playbook to automate Nginx installation, configuration and web-page deployment.
- Observed the practical significance of idempotent automation.

Conclusion:

In this assignment, Kubernetes objects and networking Services were explored and implemented using a local Docker Desktop Kubernetes cluster. The practical covered Pods, Deployments, Services, Namespaces, ConfigMaps, Secrets, persistent storage and the major Kubernetes Service types. Ansible configuration management was then demonstrated using WSL Ubuntu as the control node and two Ubuntu Docker containers as managed nodes. SSH connectivity, automated Nginx installation, custom web-page deployment, verification and repeatable configuration were successfully demonstrated. The assignment provided hands-on understanding of Kubernetes resource management and Ansible-based configuration automation within a DevOps workflow.

References:

1. Kubernetes Documentation — <https://kubernetes.io/docs/>
2. Kubernetes Services — <https://kubernetes.io/docs/concepts/services-networking/service/>
3. Kubernetes kubectl Reference — <https://kubernetes.io/docs/reference/kubectl/>
4. Ansible Documentation — <https://docs.ansible.com/>
5. Ansible Inventory Guide —
https://docs.ansible.com/projects/ansible/latest/inventory_guide/intro_inventory.html
6. Ansible Playbook Guide —
https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_intro.html
7. Docker Documentation — <https://docs.docker.com/>

Submission Links:

GitHub Repository Link: <https://github.com/SwayamMandhani06/DevOps/tree/main/Assignment-8>

Hosted / Output Link: NA – Kubernetes and Ansible practical executed locally using Docker Desktop and WSL.